

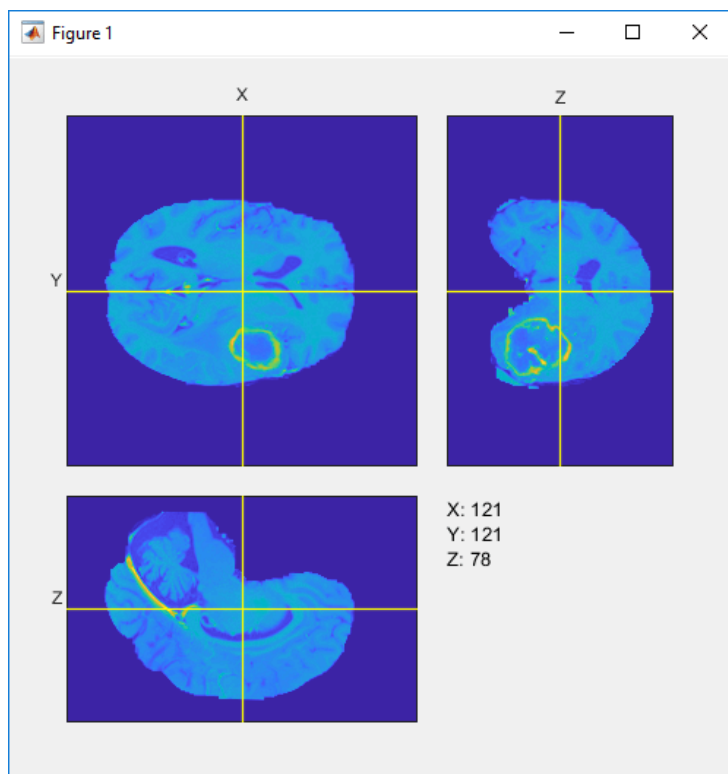
orthosliceViewer

Browse orthogonal slices in grayscale or RGB volume
Since R2019b

Description

An `orthosliceViewer` object displays volumetric image data by presenting three orthogonal views of the volume along the x , y , and z dimensions.

Use `orthosliceViewer` to look at individual slices in a volume. The `orthosliceViewer` opens, displaying the center slice in each dimension. Each view of the image stack includes a crosshair that you can use to view the different slices of the image stack. The crosshairs are linked so that if you move one, the crosshairs in the related views also move.



The `orthosliceViewer` object supports properties, object functions, and events that you can use to customize its appearance and functioning. The `orthosliceViewer` object can send notifications when certain events occur, such as the crosshair moving. For more information, see [Events](#).

i Note

By default, clicking and dragging the mouse in the slices displayed interactively changes their brightness and contrast, a technique called *window/level*. Dragging the mouse horizontally from left to right changes the contrast. Dragging the mouse vertically up and down changes the brightness. Holding down the **Ctrl** key when clicking and dragging the mouse accelerates changes. Holding down the **Shift** key while clicking and dragging the mouse slows the rate of change. Press these keys before clicking and dragging. To control this behavior, use the `DisplayRangeInteraction` property.

Creation

Description

`orthosliceViewer(V)` displays the volume V in a figure.

example

`orthosliceViewer( __, Name, Value)` sets properties using name-value pair arguments. You can specify multiple name-value pairs. Enclose each property name in single quotes.

Example: `orthosliceViewer(V, "Colormap", cmap)` creates an `orthosliceViewer` object and specifies the colormap used to display the volume.

`s = orthosliceViewer( __ )` returns an `orthosliceViewer` object, s , with properties that can be used to control the visualization of the images. Use input arguments from any of the previous syntaxes.

Input Arguments

expand all

> **V** — Input volume
numeric array

Properties

expand all

General Properties

> **Colormap** — Colormap of image stack
gray(256) (default) | *m*-by-3 numeric array

> **DisplayRange** — Display range of grayscale volume
[min(V(:)) max(V(:))] (default) | 2-element vector

> **DisplayRangeInteraction** — Interactive control of display range
"on" | "off"

> **Parent** — Parent of orthosliceViewer object
gcf (default) | uipanel | figure

> **ScaleFactors** — Scale factors used to rescale the volume
[1 1 1] (default) | 1-by-3 positive numeric vector

> **SliceNumbers** — Indices of image slices to be displayed
center slices in each orthogonal direction | 1-by-3 nonnegative numeric array

Crosshair Properties

> **CrosshairColor** — Crosshair color
[1 1 0] (default) | RGB triplet | color name | short color name

> **CrosshairEnable** — State of linked crosshair objects
'on' (default) | 'inactive' | 'off'

> **CrosshairLineWidth** — Width of crosshair line
number of points per screen pixel (default) | positive numeric scalar

> **CrosshairStripeColor** — Color of crosshair stripe
'none' (default) | RGB triplet | color name | short color name

Object Functions

addListener	Create event listener bound to event source
getAxesHandles	Get handles to axes in Orthoslice Viewer

Examples

collapse all

View MRI Data in Orthoslice Viewer

Load an image stack into the workspace.

Try This Example

 Copy Command

```
load(fullfile(toolboxdir('images'),'imdata','BrainMRILabeled','images','vol_001.mat'));
```

 Get

Create a custom Colormap.

```
cmap = parula(256);
```

 Get

View the MRI data in the Orthoslice Viewer.

```
s = orthosliceViewer(vol,'Colormap',cmap)
```

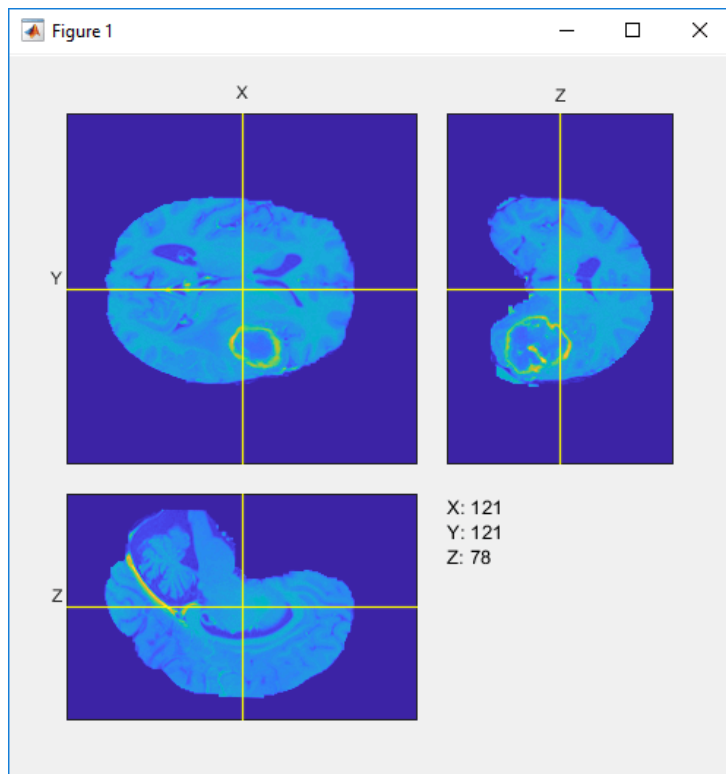
 Get

```

s =
orthosliceViewer with properties:

    SliceNumbers: [121 121 78]
    CrosshairColor: [1 1 0]
    CrosshairLineWidth: 1
    CrosshairStripeColor: 'none'
    CrosshairEnable: 'on'
    Parent: [1x1 Panel]
    Colormap: [256x3 double]
    DisplayRange: [0 2239]
    ScaleFactors: [1 1 1]
    DisplayRangeInteraction: 'on'

```



▼ Create GIF of MRI Data Slices using Orthoslice Viewer

Load MRI data and view it in the Orthoslice Viewer.

Try This Example

Copy Command

```
load(fullfile(toolboxdir('images'),'imdata','BrainMRILabeled','images','vol_001.mat'));
s = orthosliceViewer(vol);
```

Get ▾

Get the handle of the axes that contains the slice.

```
[hXYAxes, hYZAxes, hXZAxes] = getAxesHandles(s);
```

Get ▾

Turn off crosshair for better visibility.

```
set(s,'CrosshairEnable','off');
```

Get ▾

Specify the name of the GIF file.

```
filename = 'animatedYZSlice.gif';
```

Get ▾

Create an array of slice numbers in the required direction. Consider the YZ direction.

```
sliceNums = 1:240;
```

Get ▾

Loop through and create an image at the specified slice position.

```

for idx = sliceNums
    % Update X slice number to get YZ Slice.
    s.SliceNumbers(1) = idx;

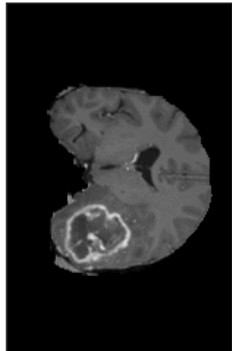
    % Use getframe to capture image.
    I = getframe(hYZAxes);
    [indI,cm] = rgb2ind(I.cdata,256);

    % Write frame to the GIF File.
    if idx == 1
        imwrite(indI,cm,filename,'gif','Loopcount',inf,'DelayTime',0.05);
    else
        imwrite(indI,cm,filename,'gif','WriteMode','append','DelayTime',0.05);
    end
end

```

 Get ▼

View the animated GIF.



▼ Set Up Listener for Orthoslice Viewer Crosshair Events

Load a stack of images.

Try This Example

 Copy Command

```
load(fullfile(toolboxdir('images'),'imdata','BrainMRILabeled','images','vol_001.mat'));
```

 Get ▼

Create a custom colormap for viewing slices.

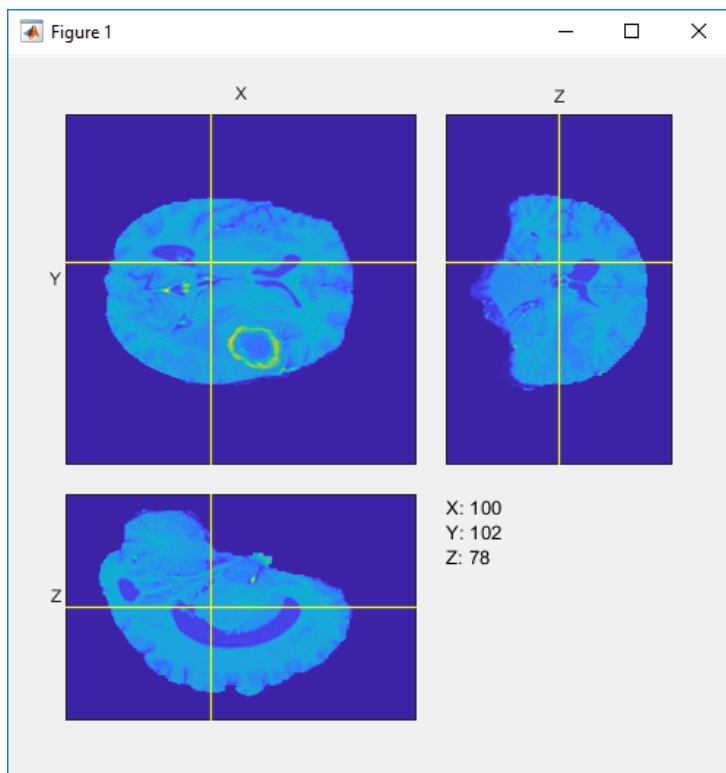
```
cmap = parula(256);
```

 Get ▼

View the image stack in the Orthoslice Viewer.

```
os = orthosliceViewer(vol,'Colormap',cmap);
```

 Get ▼



Set up listeners for the two Orthoslice Viewer crosshair moving events. When you move the crosshair, the Orthoslice Viewer sends notifications of these events and executes the callback function you specify.

```
addlistener(os, 'CrosshairMoving', @allevents);  
addlistener(os, 'CrosshairMoved', @allevents);
```

[Get](#)

The allevents callback function displays the name of each event with the previous position and the current position of the crosshair.

```
function allevents(src, evt)  
evname = evt.EventName;  
switch(evname)  
    case{'CrosshairMoved'}  
        disp(['Crosshair moved previous position: ' mat2str(evt.PreviousPosition)]);  
        disp(['Crosshair moved current position: ' mat2str(evt.CurrentPosition)]);  
    case{'CrosshairMoving'}  
        disp(['Crosshair moving previous position: ' mat2str(evt.PreviousPosition)]);  
        disp(['Crosshair moving current position: ' mat2str(evt.CurrentPosition)]);  
    end  
end
```

[Get](#)

More About

[expand all](#)

> Events

Version History

Introduced in R2019b

See Also

[sliceViewer](#) | [Volume Viewer](#) | [volshow](#) | [slice](#) | [Crosshair](#) | [obliqueslice](#)

Topics

Explore Slices from 3-D Image Volume with Anisotropic Voxel Spacing